

Transactions & locks — why money code looks the way it does

transaction.atomic

Ek business event = ek transaction. Finalize ke beech crash? PURA rollback — aadha settlement kabhi exist nahi karta. Service functions wear `@transaction.atomic`; views kabhi khud multi-row nahi likhte.

select_for_update — row lock

`SELECT ... FOR UPDATE` = “yeh row meri hai jab tak transaction khatam na ho.” Do managers ek saath finalize dabayein → doosra WAIT karta hai, phir guard use saaf mana kar deta hai (status ab draft nahi). Re-check-under-lock pattern: pehle lock, PHIR condition check (void_allocation, consume_leftover). Postgres gotcha (P1 mein mila): nullable FK ke saath select_related → LEFT JOIN → `FOR UPDATE` refuse karta hai → `select_for_update(of= ("self",))`.

Lock ORDER (deadlock se bachne ka ek hi tareeka)

Settlement hamesha isi order mein lock leta hai: `pg_advisory_xact_lock(5374)` → ADST row → stage records → worker profiles → advances. Order alag = do transactions ek doosre ka wait = deadlock. Naya money code likho to YEHI order follow karo (§11.5).

Advisory lock

`pg_advisory_xact_lock(5374)` = app-level named lock, kisi table se bandha nahi. Reference numbering (ADST-0007) ko race-safe banata hai bina counter table ke. Global key = ADR-0010 ka “one namespace” decision enforced.

What breaks without all this

Double-credit (same line paid twice), phantom drafts, deadlocked settlement day, ya — worst — aadha-likha money event jo kabhi reconcile nahi hota.